

UNIT 3: Greedy & Dynamic Programming

3.1 OPTIMIZATION.

maximum profit

minimize cost, time, distance

- Greedy involves: divide & conquer, dynamic programming, backtracking.

In each & every stage, choosing the local best feasible leads to Global Optimal.

- Dynamic: sequence of decisions for optimized and max profit.

ALGORITHM for greedy:

Alg Greedy (a, n)

for $i = 1$ to n do

$x = \text{select}(a);$

 if feasible(x) then

 { solution = solution + x; }

Applications of dynamic programming:

- optimal binary search tree
- 0/1 knapsack problem
- All pair shortest problem
- travelling salesman problem.
- reliability design.

3.2 HUFFMAN CODING.

compressing messages.

Q) Message: BCCABB DD A E C C B B A E D D C C

∴ A: 3, B: 5, C: 6, D: 4, E: 2

Arranging in ascending order; we have:

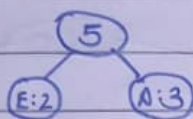
E: 2, A: 3, D: 4, B: 5, C: 6.

Now, making tree: [we make tree by choosing the first 2 smallest values, then we add them.

Small values $\rightarrow$ left side
large values $\rightarrow$ right side
if same value $\rightarrow$ left side
right $\rightarrow$ 1, left $\rightarrow$ 0

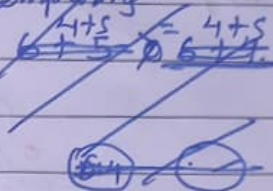
Also, for the next value, we see whether its sum with the previous sum is greater, or the sum of it with the next value. We choose the least one.]

Step 1:

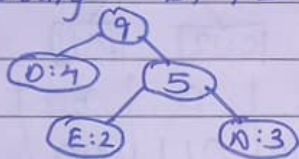


Step 2:

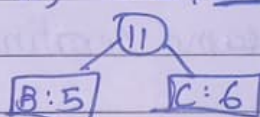
Comparing:



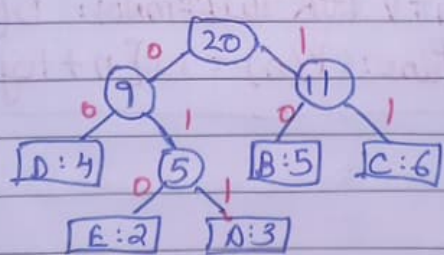
Step 2: Comparing: $5+4 = 4+5$.



Step 3: Comparing for B, $9+5 > 5+6$



Step 4:



making table: i use know 8 bits for 1 character.

Char	freq	codes	Bits
A	3	011	$3 \times 3 = 9$
B	5	10	$5 \times 2 = 10$
C	6	11	$6 \times 2 = 12$
D	4	00	$4 \times 2 = 8$
E	2	010	$2 \times 3 = 6$
<u>$5 \times 8 = 40$</u>		<u>12</u>	<u>45</u>

∴ Size of table: 40.

message size: 45

Codes: 12

∴ Total Huffman: 97 bits (ans).

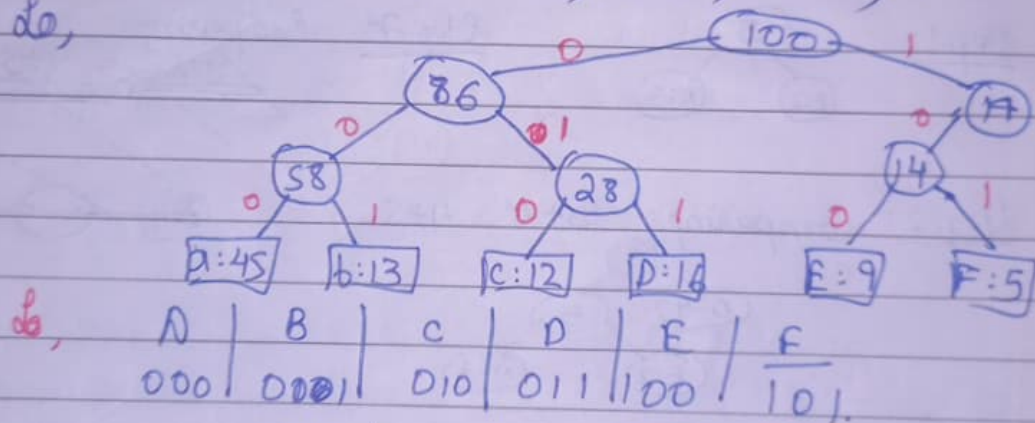
for decoding: 1 → every right side
0 → every left side.

Also, there are 2 ways for Huffman Coding:

- 1) Variable length coding (till now)
- 2) Fixed length coding (no ascending order of values)

ex: $a=45, b=13, c=12, d=16, e=9, f=5$

So,



So,

Then same continuation.

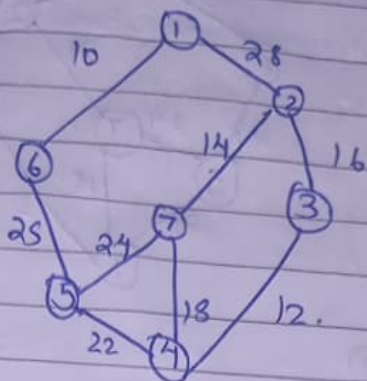
TIME COMPLEXITY FOR HUFFMAN: $O(n \log n)$

Running time func: $P(n) = N[n + \log(2n-1)] + S_n$.

3.3 MINIMUM SPANNING TREE: PRIM'S

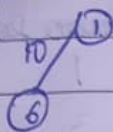
Graph represented by: $G(V, E)$.

Given Graph:



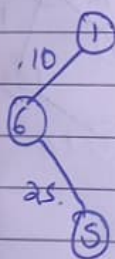
- We choose the edge with the least value, and then keep going by comparing the adjacent values.
- no circle should form.

Step 1



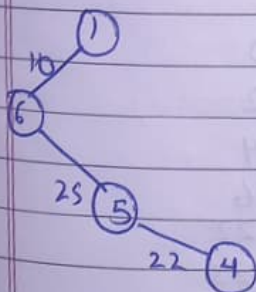
V	Known	dist	path
1	T	0	0
2	F	∞	∞
3	F	∞	∞
4	F	∞	∞
5	F	∞	∞
6	T	10	1
7	F	∞	∞

Step 2: 25 < 28



V	Known	dist	path
1	T	0	0
2	F	∞	∞
3	F	∞	∞
4	F	∞	∞
5	T	25	6
6	T	10	1
7	F	∞	∞

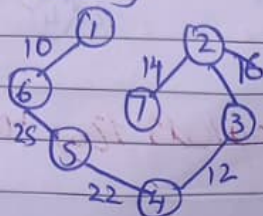
Step 3: 22, 24, 28



This table will keep going

keep comparing

finally:

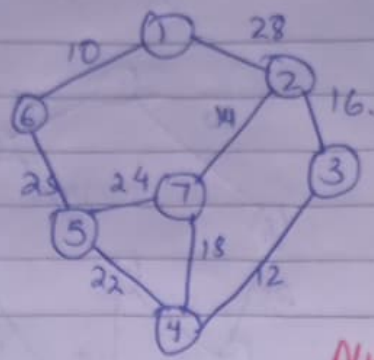


PRIM'S TIME COMPLEXITY:

$$O((V+E) \log V)$$

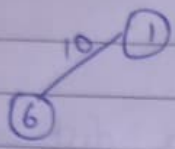
3.4 MINIMUM SPANNING TREE : KRUSKAL

Given Graph:



- Noting down every edge from least to most value.
- No cycle should form.

Step 1:

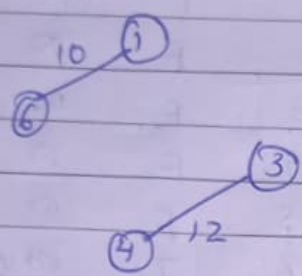


1, 6 : 10.

Also,

V. Known dist path
with every step.

Step 2:



1, 6 : 10

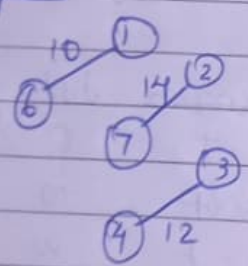
3, 4 : 12

Source Destination Wt

1 6 10

3 4 12

Step 3:

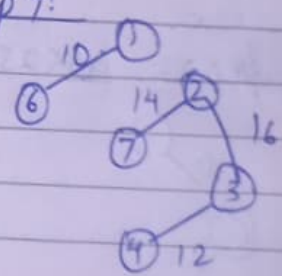


1, 6 : 10

3, 4 : 12

2, 7 : 14.

Step 4:



1, 6 : 10

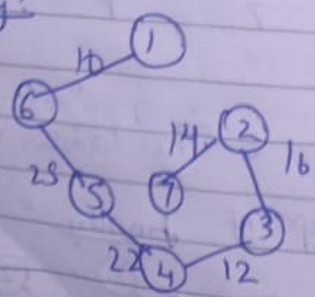
2, 7 : 14

3, 4 : 12

2, 3 : 16

Next is 7, 4 : 18, but it will create cycle.
∴ We skip it.

Going to next : 22 (5, 4) and then 25 (6, 5).
Finally:



1, 6 : 10

3, 4 : 12

2, 7 : 14

2, 3 : 16

4, 5 : 22

5, 6 : 25

Total Cost : 99.

Kruskal's T(m) : $O(E \log E)$

3.5 KNAPSACK PROBLEM GREEDY.

for divisible objects.

Q) A given bag is given with a weight.

Diff. objects are given with their weight & profit.

To include objects such as their combined weight does not exceed the bag weight, and max profit is made.

Q) Objects:

	1	2	3	4	5	6	7
P(profit):	10	5	15	7	6	18	3
W(weight):	2	3	5	7	1	4	1

bag capacity: 15.

⇒ To decide what objects go in, we need to know profit/weight.
 ∴ measuring all profits w.r.t to 1 kg.

P/W: 5 1.6 3 1 6 4.5 3

Now,

x/How much

taken from each: [1 1 1 1 1]

(container) $x_1, x_2, x_3, x_4, x_5, x_6, x_7$.

→ whole
 0 = none.

From highest to lowest P/W ratio, we put first in bag:

P/W	Object	Profit	Weight left
6	5	6.	$15 - 1 = 14$.
5	1	10.	$14 - 2 = 12$.
4.5	6	18.	$12 - 4 = 8$.
3	3	15.	$8 - 5 = 3$.
3	7	3.	$3 - 1 = 2$.
1.6	2		

→ but weight of object 2 is '3'
 and only '2' is left.

∴ We take $\frac{2}{3}$ of 3 = 2 kg. of object '2'.

∴ Profit from object 2 = $\frac{2}{3} \times 5 = \frac{10}{3} = 3.33$.

So, 1.6 2 3.33 2-2=0.
Total profit: 55.33

**Time Complexity: $O(n \log n)$
 OF KNAPSACK**

3.6 0/1 KNAPSACK PROBLEM: DYNAMIC.

→ for non divisible items.

→ tabular method

Here, since items are non-divisible, we can either take whole (1) of them, or none (0).

That's why $x = 0/1$.

while filling in the table,
an object '3' can take objects
'1' & '2' as well.

The "sum" of the max values
shall be taken.

- Q) $m = \text{capacity of bag} = 8$
 $n = 4$
 $P = \{1, 2, 5, 6\}$
 $W = \{2, 3, 4, 5\}$

Creating a table:

Step 1

			0	1	2	3	4	5	6	7	8
P	W	0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	3	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	(8)

Step 1: putting the 0th column & row with '0'
 $\therefore$ profit and weight is '0' if capacity of bag = 0

Step 2: for object '1'; weight = 2.
 $\therefore$ We fill all rows from column of weight '2',
 with its corresponding profit '1'.

Step 3: for object 3,
 profit = 2 from column '2'.
 and profit = $2 + 1 = 3$ from bag capacity =
 $5 (2 + 3)$

Step 4: for object 4; profit = 5 from C.4.
 when C6 $\rightarrow 5 + 1 = 6$, (4 + 2)
 when C7 $(4 + 3) \rightarrow 5 + 2 = 7$
 taken C8

New, taking sequence of decisions:

Finding max profit in the table,
it is 8:

π_1 π_2 π_3 π_4
1 1 0 1

$\therefore$ 4th object has to be included.

profit of 4th object: 6.

$$\therefore 8 - 6 = 2.$$

Now, profit of '2' is
found by the 2nd object
first.

$\therefore$ 2nd is included.

Total profit: $2 - 2 = 0$

Hence, only objects 2 & 4 are included.

Q2) Objects: 1 2 3 4 π_1 π_2 π_3 π_4
profit: 12 10 20 15
weight: 2 1 3 2 1 1 0 1

capacity
weight of bag: $m = 5$.

$\therefore$ table:

			0	1	2	3	4	5
P	W	D	0	0	0	0	0	0
12	2	1	0	0	12	12	12	12
10	1	2	0	10	12	22	22	22
20	3	3	0	10	12	22	22	22
15	2	4	0	10	15	25	30	37

; Here, max profit '37' found by Obj 4.

$$\text{profit left} = 37 - 15 = 22$$

22' first found by Obj 2;

$$\therefore \text{profit left} : 22 - 10 = 12.$$

'12' first found by Obj 1;

$$\therefore \text{profit left} : 12 - 12 = 0.$$

$\therefore$ Objects 1, 2, 4 are included.

TIME COMPLEXITY OF 0/1 Knapsack: $O(n \times W)$

3.7 MATRIX MULTIPLICATION:

Tells best way to multiply, not gives answer.

Intro: 2 matrices can only be multiplied if the
no. of columns of 1st matrix = no. of rows of 2nd matrix.

Ex: A and B
(5x4) (4x3)

∴ resultant matrix = 5x3.

Q) A given problem is: $A \cdot A_2 \cdot A_3 \cdot A_4$
(5x4) (4x6) (6x2) (2x7)
Find the best way to multiply them.

We make 2 tables: 'm' & 's'.

	m			
	1	2	3	4
1		0	120	88
2			0	48
3				0
4				

	s			
	1	2	3	4
1		1	1	3
2			2	3
3				3
4				

Step 1] Taking bottom-up approach, we take:

$$A \rightarrow m[1,1] \rightarrow 0$$

$$B \rightarrow m[2,2] \rightarrow 0$$

$$C \rightarrow m[3,3] \rightarrow 0$$

$$D \rightarrow m[4,4] \rightarrow 0$$

Step 2) Finding $m[1,2]$, $m[2,3]$, $m[3,4]$.

$$\text{So, } m[1,2] = m[1,1] * m[2,2]$$

$$\begin{aligned} m[1,2] &= A_1 \cdot A_2 = m[1,1] + m[2,2] + 5 \times 4 \times 6 \\ &= 0 + 0 + 120 \\ &= 120. \end{aligned}$$

∴ $k=1$

$$m[2,3] = A_2 \cdot A_3 = m[2,2] + m[3,3] + 4 \times 6 \times 2.$$

$$= 0 + 0 + 48 = 48;$$

; $k=2$

$$m[3,4] = A_3 \cdot A_4 = m[3,3] + m[4,4] + 6 \times 2 \times 7.$$

$$= 0 + 0 + 84 = 84.$$

; $k=3$

Step 2: $A_1 \cdot A_2 \cdot A_3 \rightarrow m[1,3]$

$$A_1 \cdot (A_2 \cdot A_3)$$

$$= m[1,1] + m[2,3] + 5 \times 4 \times 2$$

$$= 0 + 48 + 40$$

$$= 88$$

$$(A_1 \cdot A_2) \cdot A_3$$

$$= m[1,2] + m[3,3] + 5 \times 6 \times 2$$

$$= 120 + 0 + 60$$

$$= 180.$$

$$\therefore m[1,3] = 88.$$

; $k=1$

$$A_2 \cdot A_3 \cdot A_4 \rightarrow m[2,4]$$

$$A_2 \cdot (A_3 \cdot A_4)$$

$$= m[2,2] + m[3,4] + 4 \times 6 \times 7$$

$$= 0 + 84 + 168$$

$$= 252$$

$$(A_2 \cdot A_3) \cdot A_4$$

$$= m[2,3] + m[4,4] + 4 \times 2 \times 7$$

$$= 48 + 0 + 56$$

$$= 104$$

$$\therefore m[2,4] = 104$$

; $k=3$

Step 4: $A_1 \cdot A_2 \cdot A_3 \cdot A_4 \rightarrow m[1,4]$

do,

$$A_1 \cdot (A_2 \cdot A_3 \cdot A_4) = m[1,1] + m[2,4] + 5 \times 4 \times 7.$$

$$= 0 + 104 + 140 = 244$$

$$(A_1 \cdot A_2) \cdot (A_3 \cdot A_4) = m[1,2] + m[3,4] + 5 \times 6 \times 7$$

$$= 120 + 84 + 210$$

$$= 414$$

$$(A_1 \cdot A_2 \cdot A_3) \cdot A_4 = m[1,3] + m[4,4] + 5 \times 2 \times 7.$$

$$= 88 + 0 + 70 = 158$$

$$\therefore m[1,4] = 158; \quad k=3.$$

* Common formula : $m[i,j] = m[i,k] + m[k+1,j] + d_{i-1} \times d_k \times d_j.$

Now, by table:

~~$k=3$~~

looking at $m[1,4]$, $k=3$.

$$\therefore (A_1 \cdot A_2 \cdot A_3) \cdot A_4$$

looking at $m[1,3]$, $k=2$.

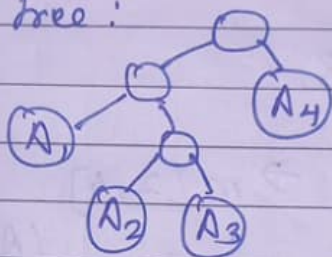
$$\therefore ((A_1 \cdot (A_2 \cdot A_3)) \cdot A_4$$

and $m[1,2]$, $k=1$:-

$\therefore$ to multiply in the form of:

$$((A_1) \cdot (A_2 \cdot A_3)) \cdot A_4$$

in the form of a tree:



TIME COMPLEXITY: $O(n^3)$

3.8 LONGEST COMMON SUBSEQUENCE

LCS using recursion

LCS using memorization

LCS using dynamic programming:

Q1) String 1: a b c d e f g h i j
String 2: c d g i

$\therefore$ cdgi is the LCS.

ex: 2: ccdgi;

here c & d come before 'e'.

$\therefore$ cdgi and eg are the common subsequences.

But,

cdgi is the longest common subsequence.

ALGORITHM:

```
int lcs [i,j]
```

```
{ if (A[i] == '10' || B[j] == '10')
```

```
    return 0;
```

```
else if (A[i] == B[j])
```

```
    return 1 + lcs(i+1, j+1);
```

```
else return max (lcs(i+1, j),
```

```
lcs(i, j+1)); }
```

here, i and j are indices of 2 strings
A and B.

- 1st Condition.- 2nd Cond.- 3rd Cond.'LCS' using Recursion.

→ top-down approach.

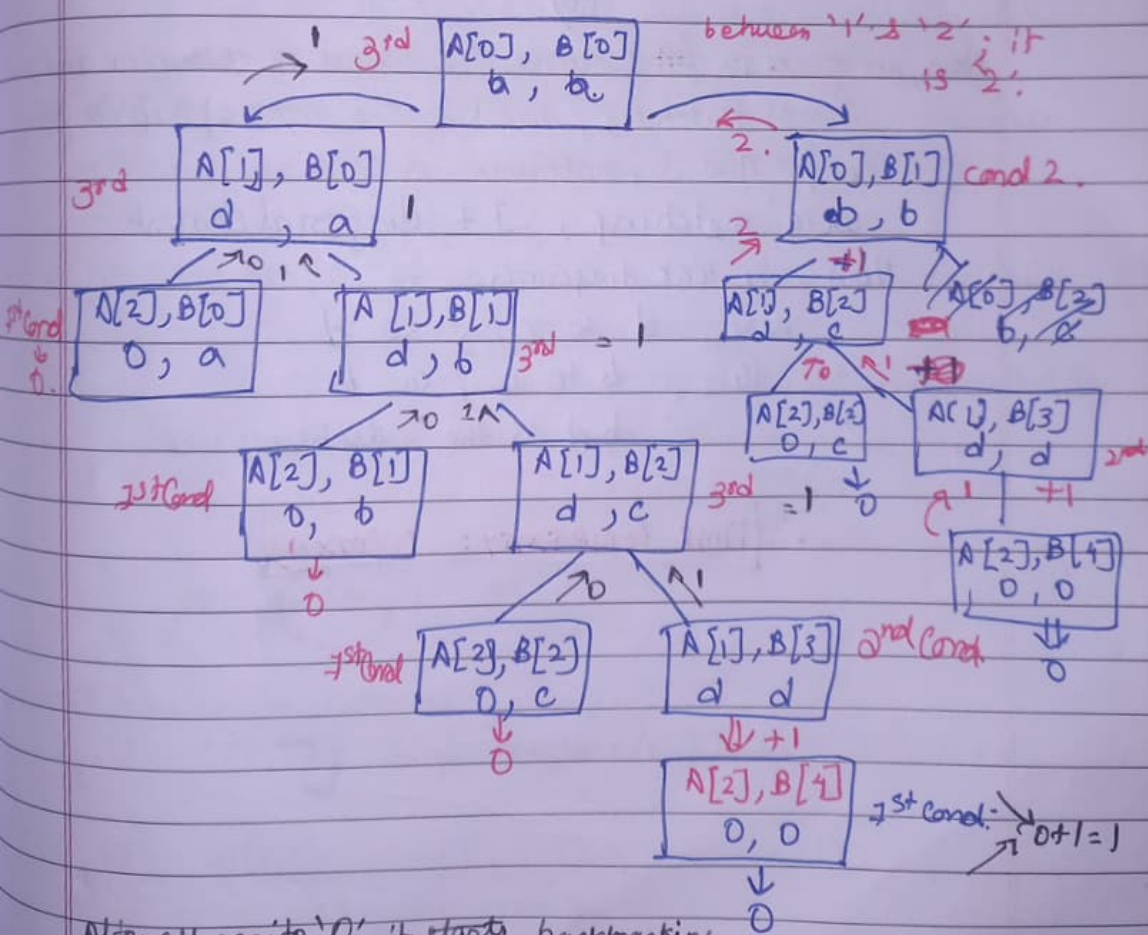
→ $P(n) = (2^{\min(m,n)})$

Q) A

b	d	10
0	1	2

 B

a	b	c	d	10
0	1	2	3	4



After all goes to '0' it starts backtracking

LCS using dynamic programming bottom-up approach

Ex) m: A

b	d
---	---

n: B

a	b	c	d
---	---	---	---

 1 2 3 4

Algorithm:

if ($A[i] = B[j]$)

$LCS[i, j] = 1 + LCS[i-1, j-1]$

else

$LCS[i, j] = \max$

$(LCS[i-1, j],$

$LCS[i, j-1])$

		a	b	c	d
	0	1	2	3	4
b	0	0	1	0	0
d	2	0	0	1	2

(b) (d)

Now, we need to ~~find~~ compare the values of columns and rows.
if not matching; we take the max of the (side) (up)
do, for row b, column a $\rightarrow 0$.
were matching, $1 + \text{diagonal element}$.

Now, we took diagonals.

Here, d to c, so d

Here, b to a, so b.

$\therefore$ bd is the subsequence.

$\therefore$ Time Complexity: $O(m \times n)$

str1: s + one
str2: longest.

∴ Table:

		L	O	N	G	E	S	T
		0	1	2	3	4	5	6
	0	0	0	0	0	0	0	0
S	1	0	0	0	0	0	0	0
T	2	0	0	0	0	0	0	0
O	3	0	0	1	1	1	1	1
N	4	0	0	1	2	2	2	2
E	5	0	0	1	2	2	3	3

(O), (N) (E)

∴ longest Common Subsequence: ONE.

3.9 OPTIMAL BINARY

SEARCH TREE (only for BST)

→ Each tree is represented as T_n ; n → number of elements.

→ binary tree → elements less than root is left side.

greater than root is right side.

→ optimal bst → for a given set of numbers, we can build multiple binary trees; but:

OBST is → minimum number of steps to reach a key

For 'n' keys, a tree is represented as T_n .

→ for n keys; no. of BSTs possible: $\frac{2n C_n}{n+1}$
; no. of unsuccessful (square) nodes: $n+1$

ex: for 3 keys, $\frac{2 \times 3 C_3}{3+1} = 5$.

#3 formulas for calculating:

$$(1) \ C(i, j) = \min \{c(i, k-1) + c(k, j)\} + w(i, j) \quad i \leq k \leq j$$

$$(2) \ W(i, j) = w(i, j-1) + p(j) + q(j).$$

p, q are probabilities.

p → successful search

q = unsuccessful search.

$$(3) \ \alpha(i, j) = k.$$